

CS 630: Software Testing and Quality Assurance

8-3 Paper: CI Monitoring Proposal

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

August 25, 2026

Continuous Integration Monitoring Proposal

Software Project

The proposed monitoring plan is for SeedShare, a fictional web application that allows members of a community gardening group to list, search for, and claim seed packets. The application includes a Vue-based user interface, a Node.js and Express application programming interface, and a PostgreSQL database. The development team consists of three full-stack developers, one quality assurance (QA) engineer, and a product owner. Team members work in two-week sprints, manage source code and pull requests in GitHub, and use GitHub Actions as the continuous integration (CI) platform. Every pull request runs installation, linting, unit tests, integration tests, and a production build in an isolated test environment before it can be merged into the main branch.

Recommended CI Metrics and Tracking

The build success rate will show the percentage of CI workflow runs that finish successfully. GitHub Actions records the result of every workflow, and a simple dashboard will summarize successful and failed runs by week and by branch. The team will review this metric during the weekly engineering meeting and examine the longer trend at the end of each sprint. An initial target of at least 90% successful main branch builds is reasonable. A falling rate may point to unstable dependencies, inconsistent local environments, or changes being submitted before developers run required checks. Test coverage will measure the percentage of application statements and branches exercised by automated tests. Jest will generate coverage reports for the backend and frontend, while a coverage service such as Codecov will display results in each pull

request and store historical trends. Coverage will be checked on every pull request and reviewed by the QA engineer for each sprint. The initial goal will be 80% statement coverage, with special attention to authentication, seed claims, inventory updates, and other high-risk functions. The team will not treat the percentage as proof that the tests are effective; reviewers must also confirm that tests cover important behaviors and failure conditions.

Mean time to repair a failed build will measure the time between a failed main branch workflow and the next successful run. GitHub Actions timestamps and issue labels will provide the required data. The product owner or QA engineer will record failures that affect the shared branch, their cause, and their resolution in the sprint tracker. This metric will be reviewed weekly, with a goal of repairing main branch failures within four working hours. Long repair times will indicate unclear ownership, difficult troubleshooting, or weak failure notifications.

Deployment frequency will measure how often SeedShare is successfully released to its staging and production environments. The deployment workflow will apply an environment label and record each completed release. The team will review staging deployments weekly and production deployments at the sprint review. The purpose is not to maximize the number of releases. Instead, the metric will show whether small, tested changes move through the pipeline regularly or remain in large batches that are harder to review and recover.

Using the Metrics for Improvement

The team will communicate results through a shared dashboard that displays current values, sprint trends, targets, and brief explanations for unusual changes. GitHub will also notify developers immediately when a required check fails. During the weekly engineering meeting, the team will discuss active failures and assign an owner. During sprint retrospectives, the team will

examine patterns across all four metrics and select one practical improvement rather than reacting to a single data point.

The results will directly influence development and QA work. Repeated build failures caused by formatting or configuration errors may lead to stronger local pre-commit checks. A coverage decline in a high-risk feature may require additional tests before a pull request can merge. Slow repair times may lead to clearer on-call ownership, better logs, or smaller pull requests. If deployment frequency decreases while work continues, the team may divide large features behind feature flags and release smaller increments. Targets will be reconsidered quarterly, so they remain challenging but realistic as the product and test suite grow.

Gaps, Limitations, and Risks

CI metrics provide useful evidence, but they do not describe the complete quality of SeedShare. A high build success rate can hide weak tests, and high coverage can include assertions that do not detect meaningful defects. Deployment frequency does not show whether users received value or experienced problems after release. Automated pipelines also provide limited information about usability, accessibility, visual consistency, security threats, and behavior in unusual browsers or devices. These areas require exploratory testing, accessibility checks, security reviews, user feedback, and production monitoring.

The plan also carries risks related to inaccurate data and unhealthy incentives. Flaky tests could make the build success rate look worse than the product quality is, while pressure to reach a coverage target could encourage low-value tests. The team will therefore document metric definitions, separate flaky-test failures from genuine defects, and interpret trends together instead

of using one number to judge individual performance. Dashboard access will be limited, and stored logs will avoid sensitive user information.